Software Security

Authentication, Authorization, and OWASP Top 10

SALINLAHI: A Voice-Interactive Filipino Language Learning Game

Systems Fundamentals — Virtualization, Containers, and Cloud Computing

Group Number: 33

Group Members:

- Daquis, Jhesza Mhei G.
- Lacida, Kylo Bryan
- Manzanero, Kyla Samantha

Course: BSIT Capstone — University of Perpetual Help System Laguna – JONELTA

Current Application Version: v0.1.0 (Android `versionCode` 2)

Repository:
`https://github.com/c23-2075-433-droid/Group33_DatabaseConnectivity`

Review Branch: `claude/capstone-security-review-f3aa1d`

Implementation Commit: `46702b1` — "Treat the save file, the key and the package as untrusted"

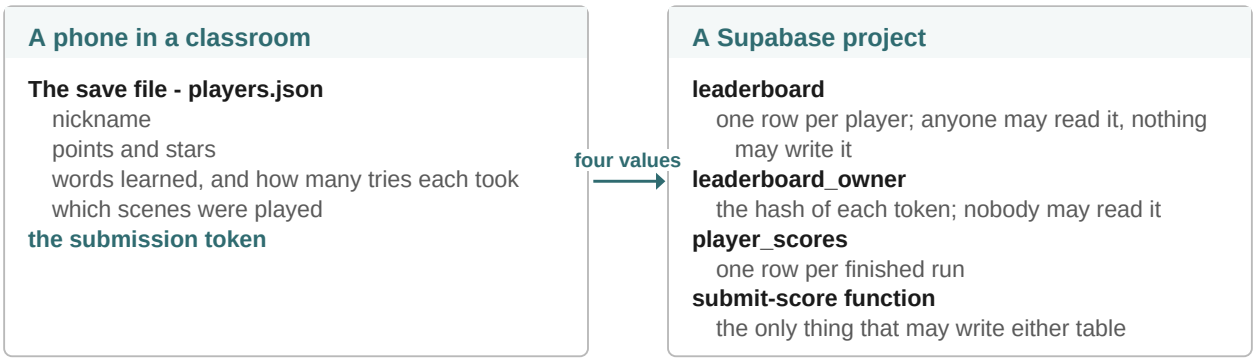
Review Date: 12 November 2026

Submitted by: Daquis, Jhesza Mhei G.

Introduction

SALINLAHI is played out loud. A child in Kinder to Grade 3 says *Bangon* and the character gets out of bed; there is no button that does it instead. The game asks for a microphone, a nickname and nothing else — no email address, no password, no birthday, no parent's name. It has no login screen and this review does not add one.

That shapes what a security review of it can honestly be. There is no account to break into. What there is, and what this review is about, is:



The leaderboard was built with this in mind, and most of it held up under testing. The parts that did not are what this document spends its length on, because a review that only lists what already works is not a review.

Three things it found are worth naming on the first page:

1. **A live API key is in this repository's history.** `Tools/elevenlabs.key` was committed on 27 September 2026 in commit `0250197` and removed from tracking two commits later, which does not remove it from history. It is reachable from `origin/main` and fourteen other remote branches. **It has to be rotated at ElevenLabs** — that is the only fix, and it is not a fix any code change in this review can perform.

2. **The score function answered a request carrying no API key at all.** Test T15 below. The token design meant a stranger still could not alter an existing player's row — and the tests show it refusing exactly that — but registering a *new* player needs no token by definition.

3. **The save file was being trusted.** It is ordinary JSON in the app's own storage, and the game read a player id out of it and spliced that id into a JSON request body by string concatenation.

All three are now addressed or, in the first case, contained and escalated.

Systems Fundamentals · Activity 9: Software Security

Part I. Security Context and Asset Identification

What this system has that is worth protecting, and from whom. Only resources that exist in SALINLAHI today are listed; there is no admin console, no teacher dashboard and no uploaded file, so none appear here.

Asset / Resource	Why It Matters	Who Should Access It?	Protection Needed
Local save file (`players.json` in `Application.persistentDataPath`) — nickname, points, stars, words learned, scenes completed, submission token	It is the child's whole record and the only copy. It also holds the token that owns their leaderboard row	The game, on this device. On Android it is in the app's private storage, so other apps cannot read it	Treated as input when read back. Not relied on for anything the server must be sure of
Submission token (one per profile, issued by the server)	The only thing that proves a score submission comes from the installation that owns the row	The device it was issued to; the server holds only its SHA-256	Never logged, never printed, never sent anywhere but the function that issued it. Stored as a hash server-side, in a table `anon` cannot read
`leaderboard` **table** — player id, nickname, two totals	It is shown to children, so a forged row is a forged thing children see	Readable by any player (that is what a board is). Writable only by the Edge Function	Row Level Security: a select policy and no insert, update or delete policy at all
`leaderboard_owner` **table** — the token hashes	Reading it would be the first step to forging submissions for every player at once	Nobody but the Edge Function's service role	RLS on with no policy, plus `revoke all ... from anon` saying the same thing a second way
`player_scores` **table** — one row per finished run: nickname, score, attempts, level	One child's results are shown to the next child who finishes a level	Insert and a narrow read by the game. No update, no delete, ever	Column-level grants and bounds on every column (added by this review)
Supabase service key	It bypasses Row Level Security entirely. With it, every protection above is decoration	The Edge Function runtime only — it is injected there and read from the environment	Never in Unity, never in the repository, never in a build. Verified by inspection
Supabase publishable key (`sb_publishable_...`)	Not a secret — it ships inside the APK by design — but it is the gate on the function	Anyone holding the app. Treated accordingly	Kept out of the repository anyway (`SupabaseConfig.asset` is git-ignored), because a key in a repository is a key nobody rotates
ElevenLabs API key	A billing credential. Leaked, somebody else spends the group's plan	The member generating voice-over, on their own machine	`Tools/elevenlabs.key`, git-ignored and read from the file or an environment variable. **Currently exposed in git history — see Part IV, finding 1**
Release keystore / APK signature	What lets a device tell the group's build from somebody else's rebuild of it	The member who produces releases	Documented as a development key today. A managed release keystore is a 1.0 requirement, not a v0.1.0 one
The microphone	The one genuinely sensitive thing the app asks for, and the mechanic itself	The game, while a child is being asked to speak	Android's runtime permission. Audio is recognised and discarded; no recording is written to disk or sent anywhere

Two things deliberately absent from this table:

There is no account and no personal data. No email address, no password, no real name, no age, no class list. A nickname a child typed is the only identifying thing that leaves the device, and the leaderboard exists to show it.

The learning record never leaves the device. Which words a child found hard, how many attempts each took, which scenes they played: that is the sensitive data in an educational game, and it is in the save file, not in Supabase. Four values go up. This was a design decision made before this review, and the review confirms the code matches it — `LeaderboardSubmission` has exactly four fields plus the token.

Part II. Authentication and Authorization Design

Does the system require user authentication?

No — and this is a design decision, not an omission.

The users are children in Kinder to Grade 3, on shared classroom tablets. A password is not an appropriate thing to ask a six-year-old for; an email address is not something they have; a Google sign-in would mean collecting a child's identity to play a vocabulary game. The game asks for a nickname and a microphone.

Every player also has identical functionality. There is no feature a teacher unlocks, no content an administrator edits, no record a parent reviews. A "teacher role" would be invented for this document and would not exist in the software, which the activity brief warns against.

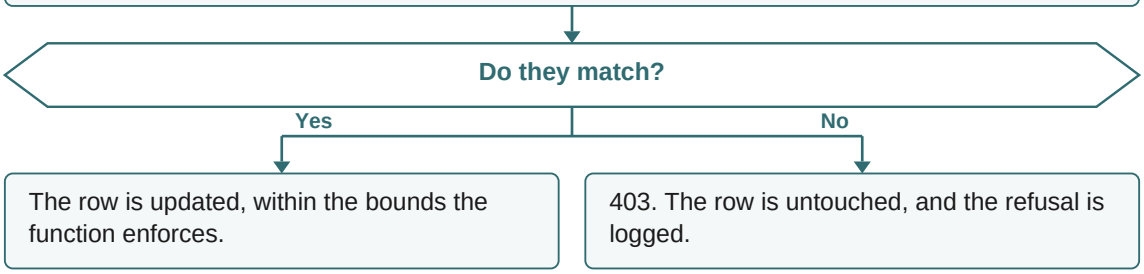
So: no user authentication.

But there is authorization, and there is a credential

This is where SALINLAHI stops being a simple case. It has no users to authenticate and it still has a resource that needs an owner: a row on a global leaderboard. Any device can claim to be any player, because a player is identified by a UUID the device made up.

The design treats that honestly:

1. The first submission for a player id registers it. No token is needed, because there is nothing yet to own.
2. The server generates a 32-byte token and returns it once, in that reply and never again.
3. It stores only SHA-256(token), in leaderboard_owner - a table the game's key cannot read.
4. Every submission after that one has to present the token it was given.
5. The function compares SHA-256 of what was sent against the stored hash, in constant time.



So **the thing being authenticated is the installation, not the person.** A player id is an identifier and identifiers are not secret; learning another child's player id buys nothing, because the token is what owns the row. That is the distinction the activity asks for, applied to a system with no users to log in: *authentication* here answers "is this the device that owns this row?", and *authorization* answers "what may that device change?" — its own row's totals,

upward, by a bounded amount, and nothing else.

Item	Design
Authentication method	A bearer token issued by the server on first submission, presented in the request body on every submission after it. No user credential of any kind
Authentication provider/service	The project's own `submit-score` Supabase Edge Function. No third-party identity provider, because there is no identity
User identity stored as	A UUID generated on the device (`Guid.NewGuid()`), kept in `players.json`. The nickname is explicitly *not* the identity, which is what lets a child be renamed without their progress moving
Session/token mechanism	No session and no expiry. The token is long-lived and device-bound: 32 random bytes, base64url, stored on the device and as a SHA-256 hash server-side. There is no refresh flow because there is nothing to refresh against
Logout / session termination	No logout — nothing is logged in. Deleting a profile forgets the token locally and the leaderboard row is deliberately left alone, because a client that could delete rows could delete anyone's

One honest limitation: **a copied save file carries the token with it.** Someone who copies `players.json` off a device onto another can submit as that player. On Android the file is in the app's private storage, so this needs either physical access with developer tools or a rooted device. For a classroom vocabulary leaderboard this is accepted rather than solved, and it is written down here rather than left for someone to discover.

Authorization Matrix

The roles are the ones that exist. "Player" is the game holding the publishable key; "Stranger" is anyone else who finds the endpoint, which the tests confirm is a real role and not a hypothetical one; "Edge Function" is the server-side code holding the service key.

Function / Resource	Player (the game)	Stranger (no app, no token)	Edge Function (service role)
Play a level, speak, earn points	Allow	n/a	n/a
Read and write its own save file	Allow	Deny (OS-enforced private storage)	n/a
Read the leaderboard	Allow	Allow — it is a public board	Allow
Read `leaderboard_owner` (token hashes)	**Deny**	**Deny**	Allow
Insert a row into `leaderboard` directly	**Deny**	**Deny**	Allow
Update or delete any `leaderboard` row directly	**Deny**	**Deny**	Allow
Submit its own score through the function	Allow	**Deny** (gated on the publishable key)	n/a
Submit a score for *another* player id	**Deny** (no token for it)	**Deny**	n/a
Register a new player id	Allow	**Deny** (gated on the publishable key)	n/a
Insert a `player_scores` row	Allow, bounded columns only	**Deny**	Allow
Read other children's `player_scores`	Allow, four columns only	**Deny**	Allow
Update or delete a `player_scores` row	**Deny**	**Deny**	Allow
Ask for its own rank	Allow	**Deny**	Allow

Where authorization is enforced

Every "Deny" above is enforced at a layer the client cannot reach:

Layer	What it enforces
Postgres Row Level Security	`leaderboard` has a select policy and no other. There is no insert, update or delete policy, so those operations are refused however the request is formed. `leaderboard_owner` has no policy at all
Postgres column privileges	`anon` may select four columns of `player_scores` and insert five. RLS decides rows; this decides columns, which is the part `select=*` was walking through
Postgres CHECK constraints	What a score, an attempt count and a nickname are allowed to be, regardless of which client sends them
The Edge Function	The token comparison, the bounds on totals, the rate limit, and the publishable-key gate. It holds the only key that may write, and it is the only writer
`security definer` **+** `set search_path = public` **on** `leaderboard_rank`	The one function `anon` may execute runs with a fixed search path, so it cannot be redirected at a shadowed table
The Unity client	**Nothing that matters.** Its validation is defence in depth and is documented as such throughout this report. An attacker edits the client; they do not edit the database

The last row is the point of the activity. Every client-side check added by this review has a server-side counterpart that was already there or was added alongside it. Nothing in SALINLAHI is protected by a hidden button.

Part III. OWASP Top 10 Applicability Review

Assessed against OWASP Top 10:2025. Every category has a justification, and four of the ten are *not* fully applicable — a game with no login and no query-building does not have the same risks as a banking portal, and saying otherwise would make this document useless as a guide to what to fix first.

OWASP Category	Applicable?	Possible Risk in Our Project	Existing / Planned Control
A01:2025 Broken Access Control	**Applicable**	A player writing the `leaderboard` table directly and skipping every check in the function. A player submitting a score for another child's player id. `player_scores` was granted table-wide select and the game asked for `select=*`, so every column of every child's run was readable with a key that ships in the APK	RLS with a select-only policy on `leaderboard`, no policy at all on `leaderboard_owner`, and the token check in the function. **Added by this review:** column-level grants on `player_scores` and a client query that names its four columns. Tested as T3, T4, T5, T10, T11, T16
A02:2025 Security Misconfiguration	**Applicable**	`verify_jwt` has to be off for the function to work with `sb_publishable_` keys, and off means the platform authenticates nobody — the endpoint answered a request with no key at all (T15). `usePlayerLog: 1` writes `Debug` output to a log on the device, and that output included the saved row and full server error bodies	**Added by this review:** the function checks the game's own publishable key itself, and the row and error bodies no longer go to the log or the screen. Verified already correct: `insecureHttpOption: 0` (cleartext HTTP refused), `enableInternalProfiler: 0`, `androidUseCustomKeystore: 0` with the development key documented in the release README
A03:2025 Software Supply Chain Failures	**Applicable**	`com.yasirkula.speechtotext` was declared as a bare git URL with no revision, so every clone, every CI checkout and every rebuild resolved to whatever that repository's default branch pointed at — including after an upstream force-push. The Edge Function imports `npm:@supabase/supabase-js@2`, a floating major that resolves at deploy time	**Added by this review:** the package is pinned to commit `27a99b18`, the revision already in use, in both `manifest.json` and `packages-lock.json`. The `supabase-js` pin is **planned, not done** — see Part IV, finding 7, and the activity's own warning about updating a dependency blindly

OWASP Category	Applicable?	Possible Risk in Our Project	Existing / Planned Control
A04:2025 Cryptographic Failures	**Applicable**	A real ElevenLabs API key is in this repository's git history and reachable from `origin/main`. Separately: tokens, if stored in clear server-side, would turn one database read into every player's credentials	**The leaked key must be rotated — see Part IV, finding 1.** Already correct: tokens are stored only as SHA-256, in a table `anon` cannot read, and compared in constant time; all traffic is HTTPS and cleartext is refused at the player-settings level. **Added:** `Tools/secret_scan.sh` and a pre-commit hook, so the next one does not get in
A05:2025 Injection	**Partially Applicable**	No SQL is built anywhere in the client — PostgREST and the RPC take parameters. But `LeaderboardClient` built the rank request's JSON body by concatenating a player id read out of the save file, which trusted that file to contain no quote, brace or backslash. A tampered save file could write its own JSON into the request	**Fixed by this review:** the body is serialized with `JsonUtility` through a typed `LeaderboardRankRequest`, and an id that is not a UUID is replaced before it is used at all. The `ilike` stem for the nickname check was already escaped with `Uri.EscapeDataString`. Tested as T17–T22
A06:2025 Insecure Design	**Applicable**	The server cannot verify a score. Nothing in the cloud hears a child say *paa* into a microphone, so "total_points: 5000" is the client's word and always will be. A design that pretended otherwise would be the insecure one	Bounded rather than solved, and documented as bounded: stars cannot exceed what the game has to give, points may only rise and by at most 1000 per submission, and one row cannot be rewritten more than once every five seconds. A forged submission can inch a score up; it cannot jump it to a million. Tested as T8, T9
A07:2025 Authentication Failures	**Partially Applicable**	There is no user authentication to get wrong — no password to be weak, no reset flow to abuse, no session to fixate. The device-level token is the authentication that exists, and the risks are its own: a guessable token, a timing-leaked comparison, a token accepted after being rejected once	32 bytes from `crypto.getRandomValues`, stored only as a hash, compared in constant time, and required on every submission after the first. Tested as T10 (wrong token) and T11 (no token). The remaining gap is the registration path, which by definition has no token — hence the publishable-key gate added here
A08:2025 Software or Data Integrity Failures	**Applicable**	A build nobody can distinguish from somebody else's rebuild. v0.1.0 is signed with Unity's development key and published as a bare APK with no checksum, so "this is the group's build" rests on where it was downloaded from. Also the unpinned dependency above: code entering the build without anyone deciding it should	The development key is documented in `Docs/release/v0.1.0/README.txt` and is appropriate for a test build. **Planned for 1.0.0:** a managed release keystore kept out of the repository, and a published SHA-256 for each release artifact. Added to the Part VII checklist. The dependency is pinned as of this review
A09:2025 Security Logging and Alerting Failures	**Applicable**	Nothing was recorded server-side. A hundred failed token checks in a minute — somebody working through player ids — and an ordinary afternoon looked identical from outside: both were silence	**Added by this review:** the function logs every refusal as one JSON line with the reason and status, and logs registrations and updates with a truncated player id, never the token and never the nickname. Visible in the Supabase dashboard's function logs. **Not added:** alerting. Nobody is paged for a capstone leaderboard, and claiming otherwise would be theatre — the honest control is that the evidence now exists to look at
A10:2025 Mishandling of Exceptional Conditions	**Applicable**	The chapter-end panel printed Supabase's reply — status code, transport error and response body — onto the screen a six-year-old was looking at. A 401 names the key; a Postgres error names a column and a constraint. Separately: a corrupt save file, a missing record, and no network at all, all of which happen on classroom tablets	**Fixed by this review:** technical detail goes to the log, and the panel says the result is safe on the device, which is true — it was written locally before the network was involved. Already correct: a corrupt save file is treated as no file rather than a crash, a failed name check lets the child through, and the board says "No connection" in a line a child can read. Tested as T12, T13, T14, T23

Two notes on what was *not* claimed:

Nothing here was invented to fill a row. A05 is "partially applicable" because the one real injection path found was a JSON body, not a SQL query, and calling it SQL injection would be wrong. A07 is "partially applicable" because there is no login to fail.

Nothing was weakened to produce a finding. Every vulnerability recorded in Part IV was already in the code before this activity began, and the git history shows it.

Part IV. Security Review of Current Implementation

Nine areas were inspected. Five is the minimum; the areas below are the ones that exist in this project rather than the ones that would round the number up.

#	Security Check	Finding	Risk / Concern	Recommended Action	
1	**Secrets in the repository — history, not just the working tree**	`Tools/elevenlabs.key` was committed on 27 September 2026 (`0250197`, 52 bytes, begins `sk_`), then removed from tracking in `2094726`. The blob is still reachable: `git branch -r --contains 0250197` lists `origin/main` and fourteen other remote branches	**The most serious finding in this review.** A billing credential that anyone who clones the repository can read. `.gitignore` never helped, because the rule was written after the file. Removing a file from tracking does not remove it from history	**Rotate the key at ElevenLabs.** That is the fix; nothing else is. Then optionally rewrite history with `git filter-repo`, which rewrites every commit hash and needs all three members to re-clone — worth doing only after rotation, and never instead of it. **Done:** `Tools/secret_scan.sh` and a pre-commit hook now refuse the next one	
2	**Secrets in the working tree and in builds**	Clean. No key, token or password in any tracked file. `SupabaseConfig.asset` is git-ignored and was never committed (`git log` on the path is empty). The service key appears nowhere in Unity, the repository or a build — the function reads it from the Edge runtime's environment	This is the part that was done right, and it is worth recording that it was checked rather than assumed	Keep `Tools/secret_scan.sh --tracked` in the release checklist. Keep documenting key *shapes* in `*.example.txt` files rather than real values	
3	**Access control beyond the UI**	`leaderboard` is select-only for `anon` and `leaderboard_owner` is unreadable, both confirmed live (T2, T3, T4, T5). `player_scores` was not: `with check (true)` on insert and table-wide select, with the game asking `select=*`	A key that ships inside the APK could insert any row into `player_scores` and read every column of every child's. Not a breach of anything secret, but it is every child's name, score, attempt count and which level they played, to anyone who unpacks an APK	**Done:** `20261112_player_scores_hardening.sql` adds bounds and narrows `anon` to four columns on select and five on insert; the client query now names those columns. **Needs running** in the SQL Editor — see Part V, improvement 5	

#	Security Check	Finding	Risk / Concern	Recommended Action	
4	**Authentication of score submissions**	The token mechanism works exactly as designed: wrong token refused (T10), missing token refused (T11), hashes unreadable (T2). But with `verify_jwt` off, a request carrying **no API key at all** reached the function and was answered (T15)	The token protects an existing row, so a stranger cannot alter a child's score. Registration needs no token by definition, so a stranger could put arbitrary rows — arbitrary nicknames — on a board children read	**Done:** `keyAccepted()` checks the publishable key the game sends. It is a gate, not authentication — the key ships in the APK — and it stops traffic that never had the app. Open until `SALINLAHI_PUBLISHABLE_KEY` is set, so deploying it cannot lock the game out	
5	**Input validation — where the boundary actually is**	The Edge Function validated properly. The client did not: `PlayerProfileStore.Clean()` only trimmed, so a nickname read from a save file could be any length and carry control characters, and the server silently kept a different string from the one the device saved. `Repair()` accepted any string as a player id and any integer as a total, including negatives	The save file is plain JSON in the app's storage. Everything read from it is input. A negative point total was read back and submitted to the board as fact; an id with a quote in it reached a hand-built JSON body (finding 6)	**Done:** `NicknameRules` holds the three rules the server applies and the client now applies them too; ids are validated as UUIDs and replaced if not; totals and star ratings are clamped on load and on write. 19 negative tests in `Tools/test_security_rules.sh`, all passing	
6	**Query and request construction**	`LeaderboardClient.RankRoutine` built its request body as `"{\"p_player_id\":\"" + playerId + "\"}"`. No SQL is built anywhere — PostgREST filters and the RPC take parameters, and the `ilike` stem is URL-escaped	A player id carrying a quote writes its own JSON into the request. The id comes from a file on the device, so this is reachable without any network attacker at all	**Done:** serialized through a typed class with `JsonUtility`, which escapes whatever it is given, and ids are refused before they get that far. Tested as T17–T22	
7	**Dependency / package review**	Two concerns, below. Nothing is outdated in a way that needs a risky upgrade; the problem is that two dependencies were not *pinned*, which is a different failure	A bare git URL resolves to the upstream default branch at fetch time. A different machine, a later clone or an upstream force-push silently changes what is in the build	`speechtotext` **pinned** to `27a99b18`. `supabase-js` pin **planned** — read the resolved version from the function's deploy output first, then pin and redeploy, rather than guessing a version and breaking the only writer the leaderboard has	
8	**Error handling and what it discloses**	`SessionScoreTracker` printed `"Could not save result.\n" + error` to the panel, where `error` was `"Save failed (401): ...`	{"message":"Invalid API key","hint":...}". SupabaseClient` also logged the whole saved row on success	Two problems in one line. A six-year-old is shown a status code and a server hint, and an attacker is handed exactly which layer refused them and why. The success log wrote the child's name and score into the device log	**Done:** `SupabaseClient.Log()` is the only place technical detail is written, and it writes to the log. Callers receive one sentence fit to display; the panel now says the result is safe on the device, which is true

#	Security Check	Finding	Risk / Concern	Recommended Action	
9	**Failure modes that are not errors**	Reviewed rather than changed. Offline is the normal state on classroom tablets: play never waits on the network, `PushIfNeeded()` retries on the next menu open, a corrupt save file is treated as no file rather than a crash, and a failed nickname check lets the child through	A game that refuses to start because a leaderboard is unreachable would be a worse failure than any of the findings above	No change. One hardening added: an unreadable `updated_at` in the rate-limit window is now refused rather than waved through, since a missing window means no rate limit at all	

Dependency register

Dependency	Purpose	Source	Current Status / Concern
`com.yasirkula.speechtotext`	Filipino speech recognition — the game's entire mechanic	`github.com/yasirkula/UnitySpeechToText.git`	**Was unpinned; now pinned** to `27a99b18ba3d10477b26b91c6c3e8f2b0c391150`, the revision `packages-lock.json` showed the project already resolved to. No version change, so nothing to be incompatible with — but it needs a Unity reimport to confirm the editor resolves the pinned form (Part V, improvement 4)
`com.unity.ugui` 2.0.0	Every menu, panel and button	Unity Registry	Current for Unity 6000.6.2f1. No action
`com.unity.ide.rider` 3.0.38, `com.unity.ide.visualstudio` 2.0.26	Editor integration only	Unity Registry	Not in the build. No action
Unity built-in modules (30+)	Audio, UI, web requests, animation	Unity Registry, versioned with the editor	Pinned by the editor version. No action
`@supabase/supabase-js@2`	The Edge Function's database client	`npm:` specifier, resolved by Deno at deploy time	**Floating major.** A redeploy can pick up a different 2.x than the one tested. Pin to the exact resolved version after reading it from the deploy output — not before
Unity 6000.6.2f1	The engine	Unity Hub	Pinned by the project. Any editor upgrade is a release decision, not a security patch
Python 3.9.6 + ReportLab 3.6.13	Documentation tooling only	macOS system Python / pip	Never shipped. No action

Part V. Implemented Security Improvements

Six improvements were implemented. Two is the minimum; these six are the ones the review found, and each is a change to the actual capstone rather than a demonstration built beside it.

Security Improvement	Risk Addressed	Component Changed	Evidence
1. Serialize the rank request; validate the player id as a UUID	A05 Injection — a hand-built JSON body carrying a value read off the device	`LeaderboardClient.cs`, `LeaderboardEntry.cs`, `PlayerProfile.cs`, `PlayerProfileStore.cs`, `NicknameRules.cs`	Commit `46702b1`; tests T17–T22

Security Improvement	Risk Addressed	Component Changed	Evidence
2. Apply the server's input rules at the input boundary, and clamp what the save file claims	A05, A10 — a nickname of any length and any character; negative totals and impossible star ratings read back as fact	`NicknameRules.cs` (new), `PlayerProfileStore.cs`	Commit `46702b1`; tests T23, T24; 19 passing assertions in `Tools/test_security_rules.sh`
3. Stop showing server error detail to the player	A10, A02 — status codes, transport errors and Postgres messages printed to a child's screen, and the saved row written to the device log	`SupabaseClient.cs`, `SessionScoreTracker.cs`, `LeaderboardClient.cs`	Commit `46702b1`; the diff of `SetText(recordsText, ...)`
4. Pin the speech-recognition package to the revision in use	A03 Supply chain — a bare git URL resolving to whatever upstream HEAD is	`Packages/manifest.json`, `Packages/packages-lock.json`	Commit `46702b1`
5. Bound and narrow `player_scores`	A01 Broken access control — `with check (true)` insert and table-wide `select=*`	`supabase/migrations/20261112_player_scores_hardening.sql` (new), `SupabaseClient.cs`	Commit `46702b1`; test T16 (before), and the SQL Editor run below
6. Gate the function on the game's key, and log what it refuses	A02 Misconfiguration, A07, A09 — a public endpoint answering with no key, and no record of any refusal	`supabase/functions/submit-score/index.ts`	Commit `46702b1`; test T15

Plus one finding that cannot be fixed by a code change and is escalated instead: **the ElevenLabs key in git history must be rotated at ElevenLabs.** The preventive control for it — `Tools/secret_scan.sh` and the pre-commit hook — is improvement 7 below.

1. The rank request was built by string concatenation

Before. `LeaderboardClient.RankRoutine` built its request body like this:

```
string body = "{\"p_player_id\":\"" + playerId + "\"}";
```

That is correct only while `playerId` contains no quote, brace or backslash. The id is read out of `players.json`, which is ordinary JSON in the app's storage — so the code was trusting a file on the device to be well-behaved. A `playerId` of `x","p_player_id":"y` produces a body with two keys in it.

Nothing in `Repair()` checked otherwise: it filled in a *missing* id and accepted any present one.

Change. A typed request class, serialized rather than spliced:

```
// Serialized, never concatenated: the id is read from a file on the
// device and so is input like any other. JsonUtility escapes it.
string body = JsonUtility.ToJson(new LeaderboardRankRequest { p_player_id = playerId });
```

And the id is validated where it enters the program, not where it is used. `NicknameRules.IsValidPlayerId()` requires the exact 8-4-4-4-12 shape `Guid.NewGuid()` writes; `Repair()` replaces anything else with a fresh UUID and clears the token alongside it, because a row can only ever have been registered under a real UUID — the function refuses anything else — so a token sitting next to a malformed id was never valid either.

After. The body cannot carry injected JSON whatever the file says, and a malformed id never reaches a request: it is replaced on load, and both the rank call and the submission refuse it independently.

Evidence. Tests T17–T22. The JSON-punctuation case is tested by name.

2. Validation at the boundary, not only at the far end

Before. `PlayerProfileStore.Clean()` was:

```
private static string Clean(string nickname) =>
    string.IsNullOrWhiteSpace(nickname) ? "" : nickname.Trim();
```

The Edge Function has stripped control characters, collapsed whitespace and capped nicknames at 16 since the leaderboard was built. The client did none of it. In ordinary play the input field's `characterLimit` hid the gap — but a nickname read back from a save file never passes through an input field, and the consequence was quiet rather than dramatic: the device kept one string, the board showed a different one, and nothing said so.

`Repair()` also accepted `totalPoints: -999` and `stars: 99` and submitted them to the board as fact.

Change. `Assets/Scripts/NicknameRules.cs` — a new file with no `UnityEngine` in it, holding the same three rules the function applies, so there is one definition on each side of the network and both are honest about which is authoritative. `PlayerProfileStore.Clean()` calls it; `Repair()` runs every nickname through it on load; totals are clamped to zero and star ratings to `LevelProgress.MaxStars`; and `RecordStars()` clamps at the setter rather than trusting its one caller.

The file is deliberately Unity-free so that `Tools/test_security_rules.sh` can compile the real rules and assert against them without opening the editor. A validation rule nobody can run a test on tends to stop being true.

After. What the device saves and what the board shows are the same string. A tampered save file cannot submit a negative total or claim more stars than the game has to give. The server still applies its own copy, because that is the one an attacker cannot skip.

Evidence. `Tools/test_security_rules.sh` — 19 assertions, all passing, listed as T17–T24 in Part VI.

3. The panel was printing the server's reply

Before. `SessionScoreTracker`, on the chapter-ending summary panel that ships in three scenes of v0.1.0:

```
SetText(recordsText, "Could not save result.\n" + error);
```

where `error` came back from `SupabaseClient` as:

```
Save failed (401): HTTP/1.1 401 Unauthorized | {"message":"Invalid API key","hint":"..."}
```

Two problems in one line: a six-year-old is shown a status code and a server hint, and anyone probing the app is told exactly which layer refused them and why. `SupabaseClient` also logged the whole inserted row on success — the child's name and score, into a log file on the device, with `usePlayerLog: 1`.

Change. One place writes technical detail, and it writes to the log:

```
private static void Log(string what, UnityWebRequest request)
{
    Debug.LogWarning("SupabaseClient: " + what + " (" + request.responseCode + "): " +
                     request.error + " | " + ...);
}
```

Callers receive one sentence fit to display. The panel now says:

```
Your result is saved on this device.
We will send it when you are online.
```

which is true — `RecordRun()` wrote it locally before the network was involved. The success log says `result saved.` and not the row.

After. Nothing on screen names a status code, a key or a column. The detail is still available to a developer with the device in front of them.

Evidence. Commit `46702b1`; the diff of all three files.

4. The speech package was unpinned

Before. `Packages/manifest.json` declared:

```
"com.yasirkula.speechtotext": "https://github.com/yasirkula/UnitySpeechToText.git"
```

A git URL with no revision resolves to the upstream default branch *at fetch time*. Three members on three machines, or one member re-cloning next month, can each get different code — and an upstream force-push would change what is in the build with no commit here to show it. This package is the game's entire mechanic: it is the microphone.

Change. Pinned to the revision the project was already using, read from `packages-lock.json`'s own `hash` field:

```
"com.yasirkula.speechtotext": "https://github.com/yasirkula/UnitySpeechToText.git#27a99b18ba3d10477b
26b91c6c3e8f2b0c391150"
```

and the same string written into `packages-lock.json`'s `version` so a fresh clone resolves to it too. This is the activity's compatibility warning taken seriously: **the version does not change.** It is the same commit, now named.

After. Every clone and every build gets the code this project was tested with, until somebody changes that line deliberately.

Verification still owed: this is the one change in this review that Unity has to confirm rather than a compiler or a test. On the next editor open, Packages should resolve `com.yasirkula.speechtotext` without re-downloading and the game should still hear speech. If the editor cannot resolve the pinned form, the revert is removing `#27a99b...` from that one line.

5. `player_scores` accepted anything and showed everything

Before. The Activity 5 table's rules, still live:

```
create policy "anon can insert scores" on player_scores
  for insert to anon with check (true);
create policy "anon can read scores" on player_scores
  for select to anon using (true);
```

`with check (true)` means the public key may insert *any* row: `score = -500`, `attempts = -1`, a `player_name` of fifty thousand characters. And the game asked for `select=*`, so every column of every child's run came back — name, score, attempt count, and which level they played — to anyone holding a key that ships inside the APK. Test T16 confirms the read worked exactly that way.

This is the ordinary case rather than an exotic one: rules written to make a feature work, never revisited to say what the feature needs.

Change. `supabase/migrations/20261112_player_scores_hardening.sql`:

- `CHECK` constraints on everything the game writes — scores and attempts between 0 and 1000, `player_name` 1–16 characters with no control characters, `level` and `remarks` length-capped. Added `NOT VALID`, which applies them to every new row while leaving rows already saved alone, because a migration that fails on live data is a migration nobody runs.
- `revoke all on public.player_scores from anon`, then column grants: `select (player_name, score, attempts, created_at)` and `insert (player_name, score, attempts, level, remarks)`. RLS decides rows; column privileges are the part of Postgres that decides columns, and that is the part `select=*` was walking through. `player_id` and `created_at` stay out of the insert grant so a client cannot choose its own id or backdate a row.
- No update and no delete, by granting neither and having no policy for either.

Alongside it, `SupabaseClient.FetchRecentScores` now names its four columns instead of `select=*` — so the client asks for the least it needs *and* the database enforces it. The client change alone would be a courtesy; the grant is what makes it a rule, and a future build that went back to `select=*` would now be refused.

After, once the migration is run. A forged insert of `score = -500` is refused by the database with a named constraint. `select=*` is refused with `permission denied for column player_id`. The summary panel's own query still works.

Status: written and committed, not yet run. It needs pasting into the Supabase SQL Editor, which is a dashboard action. Part VI marks its four tests as pending for exactly that reason, and they are written out ready to run.

6. The function answered requests with no key, and recorded nothing

Before. `verify_jwt` has to be off for this project: the game ships an `sb_publishable_...` key, which is not a JWT, so with the platform's check on, every submission gets a 401. Off is correct and documented. What was not noticed is the other half of it — off means **the platform authenticates nobody**, and the function is on a public URL. Test T15: a POST with no `apikey` header at all was answered, and ran our validation code.

The token design absorbed most of that: T10 and T11 show a stranger cannot touch a registered player's row. But registration needs no token by definition, so a stranger could register player ids and put nicknames of their choosing on a board that children read.

Separately, nothing was logged. A hundred failed token checks in a minute and an ordinary afternoon were indistinguishable from outside.

Change. `keyAccepted()` compares the `apikey` header against a `SALINLAHI_PUBLISHABLE_KEY` function secret, using the same constant-time compare as the token. Two deliberate properties:

- **It is a gate, not authentication,** and the code says so in those words. The key ships inside the APK; anyone who unpacks one can read it. What it stops is unauthenticated traffic that never had the app, which is what arrives at a public URL.
- **It fails open while the secret is unset,** logging `{"event":"gate-open"}` on every call. Deploying this code therefore cannot lock the game out of its own leaderboard; setting the secret is what switches the check on. A control that breaks the game when half-deployed is a control that gets reverted.

And `refuse()` now writes one JSON line per refusal before answering — reason and status, never the token and never the nickname — with registrations and updates logged alongside, carrying a player id truncated to eight characters: enough to tell repeated attempts on one row from attempts spread across many, without writing children's identifiers into a log in full.

One more thing was fixed here: the rate-limit window used `Date.parse(current.updated_at)` and compared the result, so an unparseable timestamp produced `NaN`, `NaN < 5` is false, and the submission went through. A missing window means no rate limit at all, so it is refused now.

After. Requests without the game's key are refused once the secret is set, and every refusal is visible in the function's logs.

Status: code committed; two dashboard steps owed — redeploy the function, and add the secret. Both are in `Docs/SupabaseSetup.md`.

7. The preventive control for finding 1

The leaked key is not fixable in code — it has to be rotated. What is fixable is that nothing was watching.

`Tools/secret_scan.sh` scans for the credential shapes this project actually handles (`sk_` ElevenLabs keys, `sb_secret_` keys, JWTs, Google keys, PEM private keys) and for filenames that are credentials whatever is inside them. It never prints a matched line: the point is to say where to look, not to copy a secret into a terminal, a log or a screenshot. Three modes: `--staged` (what the hook runs), `--tracked`, and `--history`.

`Tools/githooks/pre-commit` runs the staged scan before every commit, installed once per clone with `git config core.hooksPath Tools/githooks`. It ran on the commit that added itself — `secret_scan: clean.` appears in that commit's output — and the release process now runs `--tracked` before a build.

Verified three ways:

Mode	Input	Result
`--tracked`	the repository as it stands	`secret_scan: clean.`, exit 0
`--staged`	a planted file containing `sk_` + 40 characters	refused with the path and the rotation instruction, exit 1
`--history`	all 400+ commits, 3 minutes	found `0250197` and `2094726` — `Tools/elevenlabs.key` — exit 1

The third is the one that matters: the control finds the real leak, in this repository, without being told where it is.

Part VI. Security Testing and Negative Test Cases

Thirty-two tests. The minimum is four; the extras are there because a claim like "the token protects the row" is only worth making if somebody tried to break it.

Three kinds of test:

T1–T16 are live tests against the group's own Supabase project (`hkfxnifxkcfgelkgvnat.supabase.co`), sent with the same publishable key the game ships and the same headers `SupabaseClient.ApplyHeaders` sets. Every one of them was chosen so that **a pass writes nothing and a failure writes nothing**: the refusal tests are refused before any database write, the update test sets a column to the value it already holds, and the delete test targets a player id that does not exist. The board's one row was re-read afterwards (T6) and was untouched. Nothing outside this group's own project was tested.

T17–T24 are automated tests of the game's own validation code, run with `Tools/test_security_rules.sh`. They compile `Assets/Scripts/NicknameRules.cs` — the real shipped file, not a copy — and assert against it. They need no network and no Unity editor, so they can run on every release.

T25–T27 test the secret scanner itself, including against this repository's real history. A control nobody has tried to get past is a guess.

Access control

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T1	Read the board with the game's own query	Allowed — a leaderboard is public by design	`HTTP 200`, one row returned	**Pass**
T2	`GET /rest/v1/leaderboard_owner?select=*` — read the token hashes	Refused. These are credentials	`HTTP 401` — `permission denied for table leaderboard_owner`	**Pass**

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T3	`POST /rest/v1/leaderboard` — write the board directly, skipping the function, claiming 999,999 points	Refused by RLS. The game's key may not write this table	`HTTP 401` — `new row violates row-level security policy for table "leaderboard"`	**Pass**
T4	`PATCH /rest/v1/leaderboard?player_id=eq.<real id>` — update an existing row directly (set to the value it already held, so a success could change nothing)	Refused. No update policy exists	`HTTP 200` with body `[]` — **zero rows affected**. The row was invisible to the update, so nothing changed	**Pass**
T5	`DELETE /rest/v1/leaderboard?player_id=eq.<nonexistent id>` — delete a row directly	Nothing deleted. No delete policy exists	`HTTP 204`, no rows removed	**Pass**
T6	Re-read the board after T3, T4 and T5	The one real row unchanged in every column	Unchanged — `sugar`, 0 points, same id	**Pass**
T16	`GET /rest/v1/player_scores?select=*` — ask for every column of every child's run	Should be refused: the panel shows four columns	`HTTP 200`, **all seven columns returned** — `player_id, player_name, score, attempts, level, remarks, created_at`	**Fail — this is finding 3**
T16a	The same request, after `20261112_player_scores_hardening.sql` is run	`403` — `permission denied for column player_id`	*Pending — the migration is a dashboard action*	**Pending**
T16b	The panel's new four-column query	Allowed, and still returns what the panel needs	`HTTP 200`, exactly `player_name, score, attempts, created_at`	**Pass**

T4 is the interesting one, and it is in here as a lesson rather than a formality: **PostgREST answered `200`.** Not `403`. Row Level Security does not refuse an update it has no policy for — it makes the row invisible to it, so the update matches nothing and succeeds at changing nothing. A test that only checked the status code would have recorded this as a vulnerability. The proof is the empty array and T6.

Authentication of submissions

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T10	Submit for a registered player id with `token: "wrong-token"`, claiming 50 points and the nickname `hijacked`	Refused. The token owns the row	`HTTP 403` — `{"ok":false,"error":"That token does not own this player."}`	**Pass**
T11	The same submission with the token field absent entirely	Refused	`HTTP 403` — `{"ok":false,"error":"This player is registered; a token is required."}`	**Pass**
T15	`POST` to the function with **no** `apikey` **header at all**	Refused — nothing should answer an unauthenticated caller	`HTTP 400` — the function **ran** and answered with a validation error. The request reached our code	**Fail — this is finding 4**
T15a	The same request, once `SALINLAHI_PUBLISHABLE_KEY` is set and the function redeployed	`401` — `This request did not come from the game.`	*Pending — the secret and redeploy are dashboard actions*	**Pending**

T15 is the finding this part of the review exists for. The token design held — T10 and T11 show a stranger cannot alter a child's score even reaching the function directly — but an endpoint that answers anybody is not an endpoint

the group had decided to expose.

Input validation, server-side

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T7	`player_id: "not-a-uuid"`	Refused before any database access	`HTTP 400` — `player_id must be a UUID.`	**Pass**
T8	`total_points: -500` — the activity's own example	Refused. A score is a count of things that happened	`HTTP 400` — `total_points must be a whole number, zero or more.`	**Pass**
T9	`total_stars: 99999` — more stars than the game has to give	Refused against a real ceiling	`HTTP 400` — `total_stars is outside what the game can award.`	**Pass**
T12	`total_points: 1.5` — a fraction where a whole number belongs	Refused on type, not coerced	`HTTP 400` — `total_points must be a whole number, zero or more.`	**Pass**
T13	A body that is not JSON at all (`not json at all`)	Handled, not a crash or a stack trace	`HTTP 400` — `The body is not JSON.`	**Pass**
T14	`GET` the function instead of `POST`	Refused with the right method error	`HTTP 405` — `Use POST.`	**Pass**
T8a	`POST /rest/v1/player_scores` with `score: -500`, after the migration	`400` naming `player_scores_score_sane`	*Pending — would write a visible junk row to the live table before the migration, so deliberately not run*	**Pending**
T8b	The same insert with a 200-character `player_name`, after the migration	`400` naming `player_scores_name_sane`	*Pending, same reason*	**Pending**

T8a and T8b are written out rather than run, and the reason is worth stating: before the migration they would **succeed**. A score of -500 under the nickname of the tester's choosing would land in a live table that the chapter-end panel shows to children, and with no delete policy there is no way to take it back out. Demonstrating a vulnerability by leaving it in the game's own data is not a test, it is damage. Finding 3 is evidenced by T16 instead, which proves the same rules are wide open without writing anything.

Input validation, client-side (`Tools/test_security_rules.sh`)

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T17	`IsValidPlayerId("not-a-uuid")`	Refused	Refused	**Pass**
T18	`IsValidPlayerId("x\",\"p_player_id\":\"y")` — JSON punctuation, the exact shape the old concatenated body would have swallowed	Refused	Refused	**Pass**
T19	`IsValidPlayerId("' or 1=1 --")`	Refused	Refused	**Pass**
T20	A UUID with its hyphens removed, and a brace-wrapped UUID — formats `Guid` accepts but the game never writes	Both refused; only the game's own `D` format is accepted	Both refused	**Pass**
T21	`IsValidPlayerId("")` and the real id the game generates	Empty refused; a genuine UUID accepted, upper or lower case	As expected	**Pass**
T22	`IsValidPlayerId(null)`	Refused, not a `NullReferenceException`	Refused	**Pass**
T23	`Clean("Kyla\nSamantha")`, a NUL byte, an ANSI escape sequence, and a name of nothing but control characters	Control characters become spaces or disappear; the all-control name comes back empty	`"Kyla Samantha"`; no NUL; no escape character; `""`	**Pass**

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T24	`Clean(<500 × "A">)`, a name cut mid-word at the limit, and runs of spaces	Capped at 16, no trailing space left by the cut, runs collapsed	16 characters, trimmed, collapsed	**Pass**

```
Tools/test_security_rules.sh
  19 passed, 0 failed.
```

Secret scanning (`Tools/secret_scan.sh`)

Test	Attempt / Input	Expected Secure Behavior	Actual Result	Status
T25	Scan the repository as it stands	Clean — no credential in any tracked file	`secret_scan: clean.`, exit 0	**Pass**
T26	Stage a file containing `sk_` and 40 characters, then scan	Refused, naming the path, without printing the matched line	Refused, exit 1, path named, line not printed	**Pass**
T27	Scan all 400+ commits of history	Finds the real leaked key without being told where it is	Found `0250197` and `2094726` — `Tools/elevenlabs.key`	**Pass (and the finding is real)**

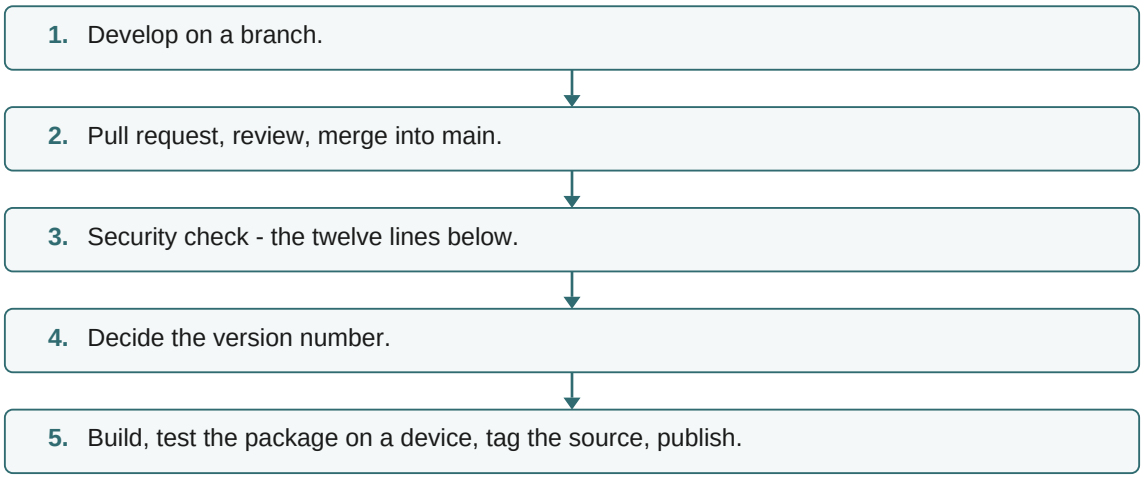
Summary

	Count
Passed	26
Failed — T16 and T15, which are findings 3 and 4. Both are addressed in code; both need the dashboard step in "What is still owed" before they can be re-run as passes	2
Pending a dashboard action — T15a and T16a wait on the secret and the migration; T8a and T8b additionally cannot be run before the migration without writing junk into live data	4
Total	32

Both failures were found by this review, in code that predates it, and neither was manufactured: `git log` shows `player_scores`'s policies unchanged since Activity 5 and the `verify_jwt` setting unchanged since the function was first deployed.

Part VII. Security Checklist for Future Releases

This replaces nothing in `Docs/RELEASE_PROCESS.md`; it slots into it. The release loop established in Activity 8 now reads:



Two of the ten lines are commands rather than judgement, and that is deliberate: a checklist whose every line needs a careful human is a checklist that gets skimmed.

Security Release Check	How	Done?
1. No credential in the release or the repository	`Tools/secret_scan.sh --tracked` — exit 0 required	
2. Input validation rules still hold	`Tools/test_security_rules.sh` — 19/19 required	
3. The pre-commit hook is installed on every member's clone	`git config --get core.hooksPath` returns `Tools/githooks`	
4. Every key the release depends on has been rotated since any exposure	ElevenLabs, Supabase publishable. A key that has ever been committed counts as exposed	
5. Database permissions reviewed against the queries the build actually makes	Every `select=` in `Assets/Scripts` names its columns, and `anon` is granted no more than those	
6. Authorization restrictions re-tested against the live project	T2, T3, T4 and T10 from Part VI. They write nothing, so they are safe to re-run every release	
7. Dependencies pinned to a revision, not a branch	No bare git URL in `Packages/manifest.json`; `supabase-js` pinned to an exact version in the function	
8. Debug and development configuration reviewed	`insecureHttpOption: 0`, `enableInternalProfiler: 0`, and no `Debug.Log` that writes a player's row or a server error body	
9. Error messages reviewed for what they disclose	Nothing shown to a player names a status code, a key, a column or a constraint	
10. Release tested with invalid and unexpected input	At least: a nickname at and past 16 characters, a level finished with no network, and the game launched with a hand-edited save file	
11. Known security issues documented in the release notes	Including the ones accepted rather than fixed — a copied save file carries its token, and the server cannot verify a score	
12. The release artifact can be identified	For 1.0.0: a release keystore kept out of the repository, and a published SHA-256 for the APK. For v0.x, the development key is documented and that is enough	

Lines 4, 11 and 12 are the ones this review added that a generic checklist would not have: line 4 because of what Part IV found in this repository's history, line 11 because two real limitations are accepted rather than solved and a release that hides them is dishonest, and line 12 because an unsigned build is a build nobody can authenticate.

Relevant Screenshots / Technical Evidence

Everything in this report can be reproduced from the repository. No screenshot is needed to read any of it, which is why the evidence below is commands and their output rather than pictures of them.

The leaked key, found by the control that now guards against it

```
$ Tools/secret_scan.sh --history
Scanning every commit. This takes a moment.
  20947266a3379182f5b9919135cbbc7d74359c20  Tools/elevenlabs.key  (a credential by filename)
  02501975f3a7b568d42c2d59aa97ac23dfc4b1b3  Tools/elevenlabs.key  (a credential by filename)

secret_scan: the files above look like they carry a credential.
...
exit 1

$ git log --all --oneline -- Tools/elevenlabs.key
2094726 Stop tracking the ElevenLabs key
0250197 Create elevenlabs.key

$ git branch -r --contains 0250197 | wc -l
      15
```

Fifteen remote branches, `origin/main` among them. The blob is 52 bytes and begins `sk_`. It is not reproduced here, in a screenshot, or anywhere else in this document.

The validation tests

```
$ Tools/test_security_rules.sh
SALINLAHI security rules - negative tests

Nickname length (cap is 16)
  PASS  a 500-character nickname is capped, not stored
  PASS  a nickname inside the cap is left alone

Control characters
  PASS  a newline becomes a space, so two words stay two words
  PASS  a NUL byte does not survive
  PASS  an ANSI escape sequence loses its escape character
  PASS  a nickname of nothing but control characters comes back empty
  PASS  surrounding whitespace is trimmed
  PASS  a run of spaces collapses to one
  PASS  null is handled rather than thrown on
  PASS  a name cut at the limit does not end in a space

Player id (must be the UUID the game made)
  PASS  the game's own id shape is accepted
  PASS  upper case hex is still a UUID
  PASS  a hand-typed id is refused
  PASS  an id carrying JSON punctuation is refused
  PASS  an id with SQL in it is refused
  PASS  a UUID without its hyphens is refused
  PASS  a braced UUID is refused
  PASS  an empty id is refused
  PASS  a null id is refused rather than thrown on

19 passed, 0 failed.
```

The live refusals

```
T2  GET  /rest/v1/leaderboard_owner?select=*
    HTTP 401  permission denied for table leaderboard_owner

T3  POST /rest/v1/leaderboard  {"total_points":999999,...}
    HTTP 401  new row violates row-level security policy for table "leaderboard"

T10 POST /functions/v1/submit-score  {"token":"wrong-token","nickname":"hijacked",...}
    HTTP 403  {"ok":false,"error":"That token does not own this player."}

T11 POST /functions/v1/submit-score  (no token field)
    HTTP 403  {"ok":false,"error":"This player is registered; a token is required."}

T8  POST /functions/v1/submit-score  {"total_points":-500,...}
    HTTP 400  {"ok":false,"error":"total_points must be a whole number, zero or more."}

T9  POST /functions/v1/submit-score  {"total_stars":99999,...}
    HTTP 400  {"ok":false,"error":"total_stars is outside what the game can award."}

T15 POST /functions/v1/submit-score  (no apikey header at all)
    HTTP 400  {"ok":false,"error":"player_id must be a UUID."}   <- the function ran
```

The two findings, side by side

```
T16 GET /rest/v1/player_scores?select=*
    HTTP 200  columns returned:
      ['attempts','created_at','level','player_id','player_name','remarks','score']

T16b GET /rest/v1/player_scores?select=player_name,score,attempts,created_at
    HTTP 200  [{"player_name":"rc","score":6,"attempts":6,"created_at":"..."}]
```

Seven columns where the panel shows four. The second query is what the game asks for now; the migration is what makes the first one stop working.

The hook, on the commit that added it

```
$ git commit
secret_scan: clean.
[claude/capstone-security-review-f3aa1d 46702b1] Treat the save file, the key and the package as unt
rusted
```

The code still builds

Both script assemblies compile clean with warnings as errors, outside the editor, using the Roslyn compiler inside the Unity 6000.6.2f1 install:

```
Assets/Scripts/*.cs                    -> exit 0
Assets/Scripts/*.cs + Assets/Editor/*.cs -> exit 0
```

Files changed

File	What changed
`Assets/Scripts/NicknameRules.cs`	**New.** The validation rules, with no `UnityEngine` in them so they can be tested
`Assets/Scripts/LeaderboardClient.cs`	Serialized rank request; id validated before submitting; error detail to the log only
`Assets/Scripts/LeaderboardEntry.cs`	`LeaderboardRankRequest`, so the body is serialized rather than spliced
`Assets/Scripts/PlayerProfile.cs`	`HasValidId()` / `IsValidId()`

File	What changed
`Assets/Scripts/PlayerProfileStore.cs`	Ids validated and replaced; nicknames cleaned on load; totals and stars clamped
`Assets/Scripts/SupabaseClient.cs`	One `Log()` for technical detail; child-safe strings to callers; named columns instead of `select=*`
`Assets/Scripts/SessionScoreTracker.cs`	The panel no longer prints the server's reply
`supabase/functions/submit-score/index.ts`	Publishable-key gate, refusal logging, rate-limit window guard
`supabase/migrations/20261112_player_scores_hardening.sql`	**New.** Bounds and column grants for `player_scores`
`Packages/manifest.json`, `packages-lock.json`	Speech package pinned to `27a99b18`
`Tools/secret_scan.sh`, `Tools/githooks/pre-commit`	**New.** The control for finding 1
`Tools/test_security_rules.sh`, `Tools/securitytests/SecurityRuleTests.cs`	**New.** 19 negative tests, no Unity required
`Docs/SupabaseSetup.md`	The function secret, the hardening migration, three new troubleshooting rows
`Docs/RELEASE_PROCESS.md`	The security check, between the merge and the version bump

What is still owed

Three things, none of which this review could do from a repository:

1. **Rotate the ElevenLabs key.** Finding 1. The only fix.
2. **Run** `20261112_player_scores_hardening.sql` in the Supabase SQL Editor, then re-run T16a, T8a and T8b.
3. **Redeploy** `submit-score` **and set** `SALINLAHI_PUBLISHABLE_KEY`**,** then re-run T15a. Setting the secret is what switches the gate on; until then it logs that it is open.

And one thing the next editor session should confirm: that Unity resolves the pinned speech package without re-downloading it.

Part VIII. Individual Security Contribution

Student Name: Daquis, Jhesza Mhei G.

Group Number: 33

Security Area Reviewed: Secrets, release configuration, and the security gate in the release process — what the group ships, and what must not be inside it.

Component / File / Configuration:

- `Tools/secret_scan.sh`, `Tools/githooks/pre-commit` (new)
- `.gitignore`
- `ProjectSettings/ProjectSettings.asset`
- `Docs/RELEASE_PROCESS.md`
- Part VII of this report

Specific Contribution

My release responsibility from Activity 8 was the build and the package, so the question I took was the one a release has to answer: is there anything in this repository that should not leave the group's machines?

– **The finding that mattered most in this review.** A real ElevenLabs API key was committed on 27 September 2026 in `0250197`, 52 bytes beginning `sk_`, and removed from tracking two commits later in `2094726`. Removing a file from tracking does not remove it from history: `git branch -r --contains 0250197` lists `origin/main` and fourteen other remote branches, so the key is in every clone. `.gitignore` was never going to help, because the ignore rule was written after the file already existed. **This is escalated rather than fixed: the key has to be rotated at ElevenLabs,** and no change in this repository substitutes for that.
– **A control, so the next one does not get in.** `Tools/secret_scan.sh` looks for the credential shapes this project actually handles — ElevenLabs `sk_` keys, Supabase `sb_secret_` keys, JWTs, Google keys, PEM private keys — and for filenames that are credentials whatever is in them. It never prints a matched line, because the point is to say where to look, not to copy a secret into a terminal or a screenshot. `Tools/githooks/pre-commit` runs it on every commit; it ran on the commit that introduced it.
– **I tested the control three ways** rather than trusting it: clean on the repository as it stands, refusing a planted `sk_` file with a named path and exit 1, and run across all 400-plus commits, where it found the real leak without being told where it was (T25, T26, T27).
– **Verifying what the build is configured to do.** `insecureHttpOption: 0`, so cleartext HTTP is refused and every Supabase call has to be HTTPS; `enableInternalProfiler: 0`; the service key absent from Unity, the repository and the build, which I checked by scanning rather than assuming. One finding: `usePlayerLog: 1` writes `Debug` output to a log on the device, and that output included the whole saved row and full server error bodies — which is what improvement 3 took out.
– **The APK's integrity.** v0.1.0 is signed with Unity's development key and published as a bare APK with no checksum, which the release README already states. It is appropriate for a test build and not for 1.0.0, so a managed release keystore kept out of the repository and a published SHA-256 are now line 12 of the checklist rather than something to remember.
– **Where the security check sits in the release loop.** Between the merge and the version bump, in `Docs/RELEASE_PROCESS.md` section 2b — the point where there is still time to fix what it finds without unpicking a release. Two of the ten lines are commands with a required exit status, because a checklist whose every line needs careful human judgement is one that gets skimmed.

Security Improvement or Test Performed

Improvement 7 in Part V (the secret scanner and the pre-commit hook), the release-configuration half of the Part IV review, tests T25–T27, and Part VII.

Repository / Changeset / Commit Evidence

Item	Reference
Implementation commit	`46702b1` "Treat the save file, the key and the package as untrusted" on `claude/capstone-security-review-f3aa1d`
The leak	`0250197` "Create elevenlabs.key"; `2094726` "Stop tracking the ElevenLabs key" — still reachable from `origin/main`
The control	`Tools/secret_scan.sh`, `Tools/githooks/pre-commit`
The hook running	`secret_scan: clean.` in the output of commit `46702b1`
Release process	`Docs/RELEASE_PROCESS.md` section 2b
Prior ownership of this area	Release preparation, build generation and packaging for v0.1.0, per `Docs/RELEASE_PROCESS.md`'s responsibility table

What security consideration will I personally check while developing succeeding features?

Whether the feature needs a credential, and if it does, where that credential lives when I am not looking at it. Every secret this group has handled got into trouble the same way: it was convenient to put it in a file, and the file was in the project folder. So before a feature that talks to anything — a text-to-speech service, an analytics endpoint, a teacher's portal — I will decide where the key goes before I write the call that uses it, add the path to `.gitignore` in the same commit, and run `Tools/secret_scan.sh` before pushing. And if a key ever does get committed again, I will rotate it the same day rather than deleting the file and hoping.